

Resumo

Slide 1- Introdução

- Primeira vez que o termo de engenharia de software foi usado → **conferência da OTAN**

Definição de engenharia de software

- Área da computação destinadas a investigar os desafios e propor soluções que permitem desenvolver sistemas de software

(01) Requisitos de software

- **O que** o sistema deve fazer e **como** para atender aos seus clientes com qualidade de serviço

REQUISITOS FUNCIONAIS

- “O que” um sistema deve fazer
 - Funcionalidades ou serviços que ele deve implementar
 - Exemplo: informar saldo da conta, realizar transferências ...

REQUISITOS NÃO FUNCIONAIS

- “Como” o sistema deve operar
 - Sobre quais restrições e com qual qualidade de serviço
 - Exemplo: desempenho, disponibilidade, capacidade, tolerância a falhas, segurança, privacidade interoperabilidade, manutenibilidade, usabilidade ...

(02) Projeto de software

- Definição das principais unidades de código de um sistema

(03) Construção de software

- Implementação (codificação) do sistema

(04) Testes de software

- Verificam se um programa apresenta um resultado esperado ao ser executado com casos de teste

Verificação: estamos implementando o sistema corretamente?

- De acordo com os requisitos e especificação

Validação: estamos implementando o sistema correto?

- O sistema que os clientes querem

(05)Manutenção de software

- **tipos:** Corretiva, Preventiva, Adaptativa, Refactoring, Evolutiva.

REFACTORING

- Manutenção para melhorar a manutenibilidade, ou seja, não corrige bugs, não implementa novas funcionalidades:
 - Renomear variáveis, extrair funções, mover funções ...

(06)Gerência de configuração

- Todo software é desenvolvido usando um sistema de controle de versões
- Permite recuperar versões antigas

VERSIONAMENTO SEMÂNTICO (RELEASE X.Y.Z)

- X: versão com adição de novas funcionalidades muito diferentes da versão anterior (major version)
- Y: versão com adição de novas mas pequenas funcionalidades (minor version)
- Z: versão com apenas correções de bugs (normalmente chamada de patch)

(07)Gerência de projetos

- Negociação de contratos, gerência de RH, gerenciamento de riscos, acompanhamento da concorrência, marketing ...

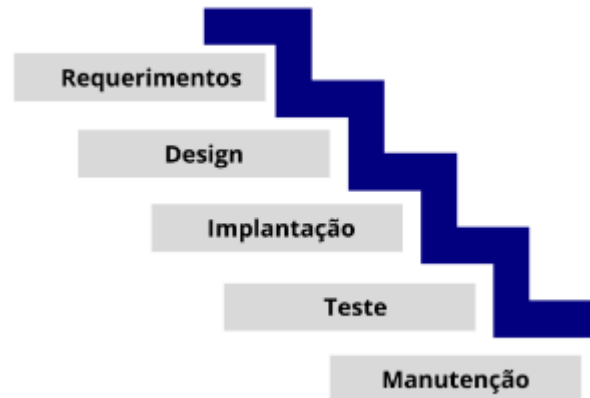
Stakeholders: Todas as partes interessadas em um projeto de software, desde aqueles que afetam ou são afetados pelo projeto

(08)Processos de desenvolvimento de software

- Define as atividades e etapas que devem ser seguidas para construir um sistema de software

MODELO CASCATA

- Inspirado em processos de engenharias tradicionais, como Civil, Mecânica, Elétrica, etc



Problemas:

- Requisitos mudam com frequência
 - Levantamento completo de requisitos demanda tempo
 - Quando ficar pronto, o "mundo já mudou"
 - Clientes às vezes não sabem o que querem
- Documentações de software são extensas
- E rapidamente se tornam obsoletas

MANIFESTO ÁGIL

- Modelo incremental e iterativo

(09)Modelagem de software

- Representação de mais alto nível do que o código
- Utiliza-se UML

(10)Qualidade de software

- Qualidade Externa:
 - Correção, robustez, extensibilidade, reusabilidade, eficiência, compatibilidade, facilidade de uso, portabilidade
- Qualidade Interna:
 - Modularidade, legibilidade, testabilidade, manutenibilidade, etc

Estratégias para garantia de qualidade de software: Métricas e revisões de código

(11)Prática profissional

(12)Aspectos econômico

Tipos ABC de sistemas

SISTEMA C (CASUAIS)

- Tipo muito comum de sistema
- Sistemas pequenos, sem muita importância

SISTEMAS B (BUSINESS)

- Sistemas importantes para uma organização

SISTEMAS A (ACUTE)

- Sistemas onde nada pode dar errado, pois o custo é imenso, em termos de vidas humanas e dinheiro

Slide 2- Introdução

Microempresas de Software como Serviço

- **Micro-SaaS** é um pequeno empreendimento de software, autofinanciado, administrado por um ou dois desenvolvedores e que atende a uma necessidade específica
- Micro-SaaS segue um modelo de serviço (**Software as a Service**), com pagamentos recorrentes (assinaturas) por parte de centenas ou alguns clientes
- Paga-se uma assinatura → usuário ganha acesso ao sistema → empresa vendedora matem toda a infraestrutura

MÉTRICAS PARA AVALIAR SUCESSO DOS SAAS

- Custo de aquisição de clientes (CAC):
 - custos com propaganda, vendas, etc em um mês / número de novos assinantes no mês
- Receita mensal recorrente (MMR)
- Taxa de cancelamento
- Valor do tempo de vida do cliente

Para um SaaS ser viável $LTV > CAC$

- Net promoter score (NPS): Avaliação dos clientes

Precificação de Produtos de Software

- O pagamento dá o direito de instalação e receber atualizações
- Podem ser individuais ou compartilhada

| Sistemas freemium: Oferecem funcionalidades ou planos gratuitos

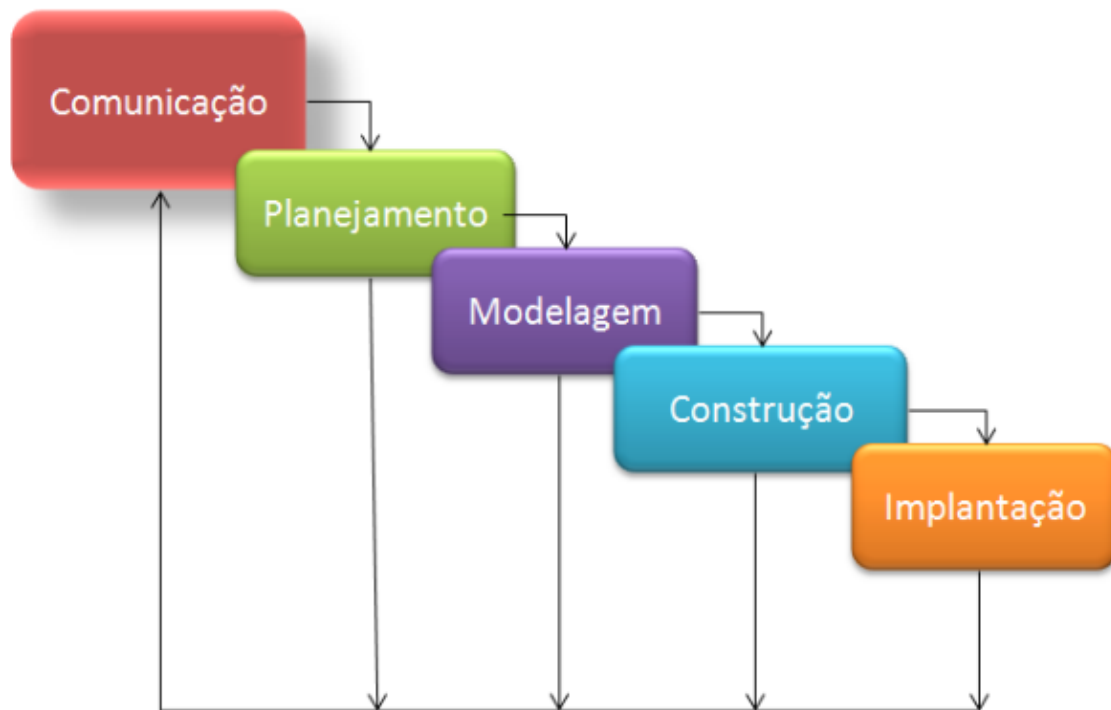
Slide 3- Processos Software

- Um processo define quem está fazendo o que, quando e como para atingir certo objetivo

Vantagens em se ter um modelo de processo:

- O tempo de treinamento pode ser reduzido
- Produtos podem ser uniformizados
- Resultados previsíveis e gerenciáveis
- Possibilidade de capitalizar experiências

Modelo de cascata (Modelo sequencial)

**Comunicação:**

- Stakeholders e organização-alvo, elicitação de requisitos

Planejamento:

- Escopo do produto a ser desenvolvido, riscos, plano de trabalho, recursos

Modelagem:

- Análise requisitos, projeto do sistema, camada de domínio, camada de interface

Construção:

- Geração de código e testes

Implementação:

- Operação do produto, feedback usuários

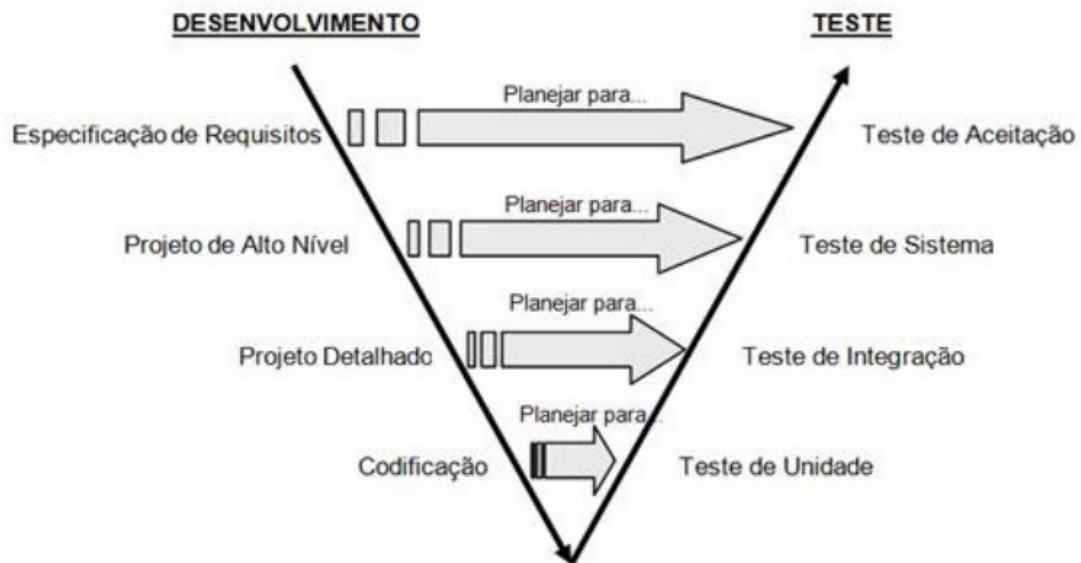
CONTRIBUIÇÕES

- Organização de atividades, disciplina, planejamento e gerenciamento
- Documentação

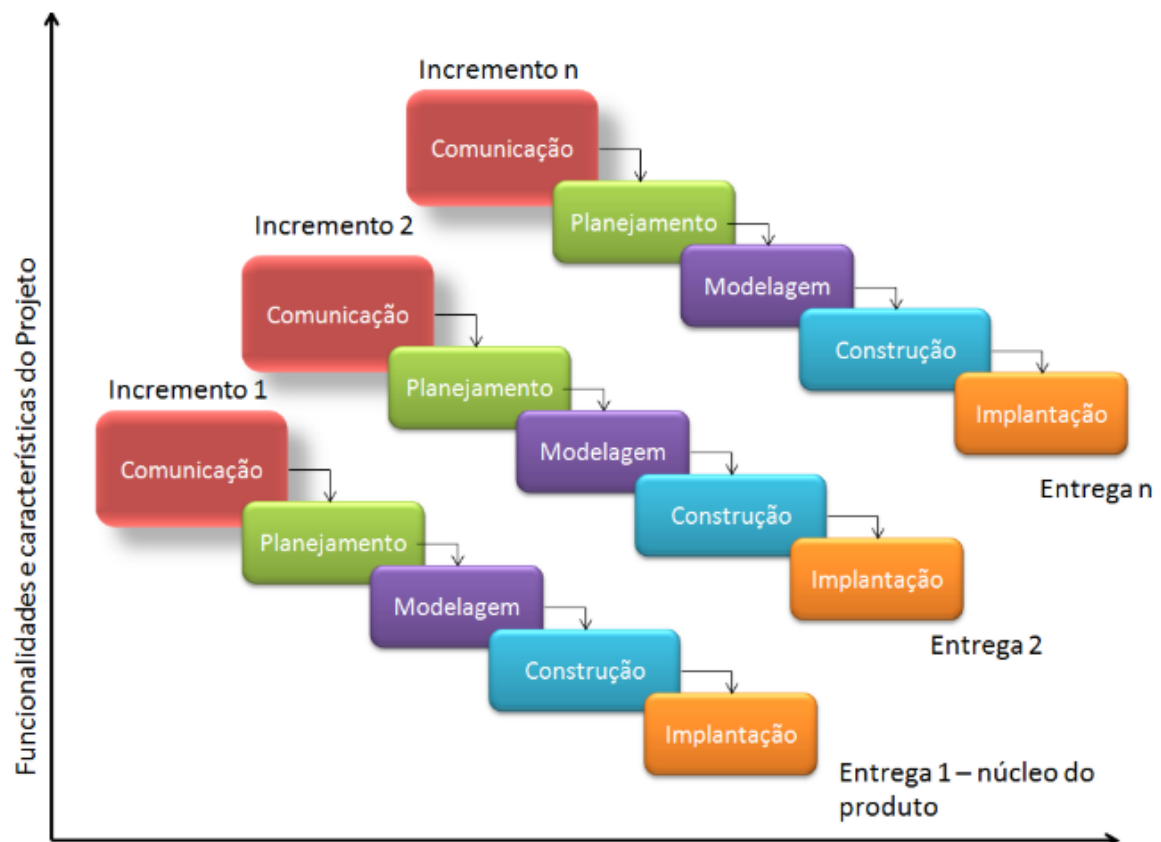
PROBLEMAS

- Difícil elicitar todos os requisitos logo no início
- Projetos reais raramente seguem o fluxo sequencial que o modelo propõe
- Uma versão executável do software só fica disponível após a construção

Modelo de cascata V e W (Foco em testes)



Modelo incremental



Modelo Evolucionários

- Tratam a natureza evolutiva do software
 - Requisitos instáveis
 - O conteúdo funcional de um sistema deve crescer continua e progressivamente
- Versões progressivamente mais completas
- Mitigação de riscos



PROBLEMAS

- Gerenciamento complexo: número incerto de ciclos
- Rápido? / lento?
- Foco principal na extensibilidade e não em qualidade

Métodos Ágeis

Exemplos: Scrum, XP, Crystal, FDD, DSDM, ASD

CARACTERÍSTICAS

- Equipes e processos auto-ajustáveis
- Interativo e incremental
- Orientado a cronogramas breves
- Pouca documentação
- Programação em pares

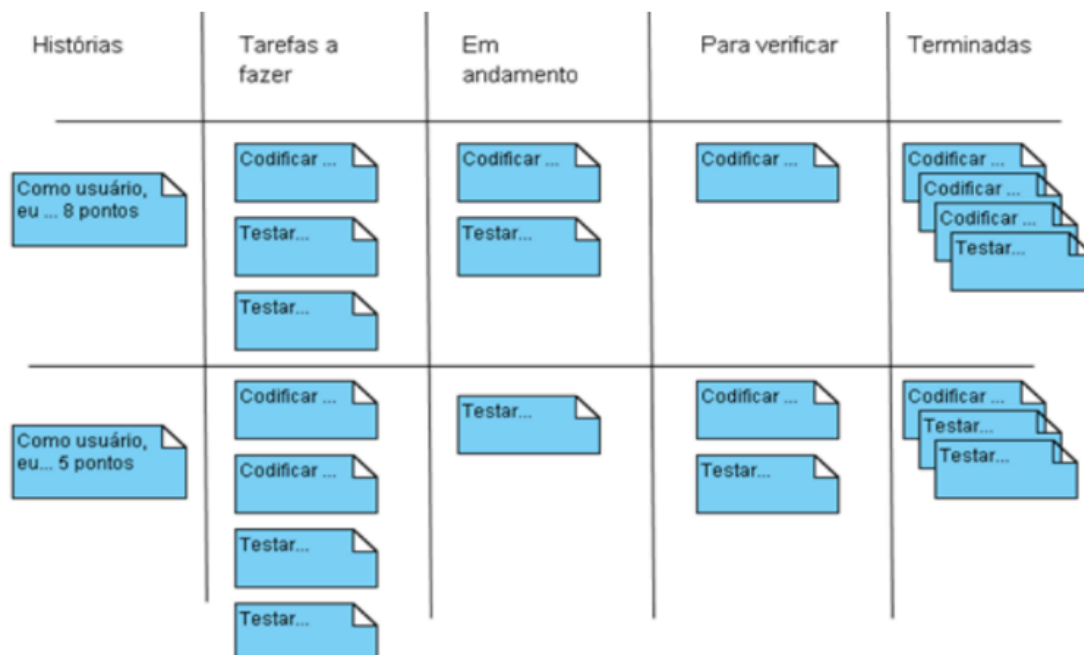
- Código coletivo
- Não orientados a ferramentas
- Desenvolvimento dirigido por testes

Scrum

Ciclo de vida



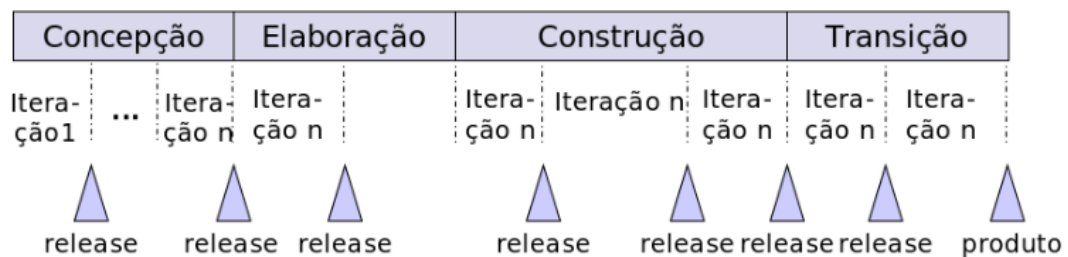
Scrum board



Processo unificado

- Forte interação com usuário

- Dirigido por casos de uso
- Baseado em componentes
- Centrado na arquitetura
- Iterativo e incremental



- Cada iteração gera um artefato ou conjunto deles (release)

Concepção:

- Define o escopo do projeto e sua viabilidade
- Deve ser breve: 2 semanas a 2 meses
- Marco: Lifecycle Objective Milestone (LCO)

Elaboração:

- Detalhar a análise de requisitos (80%)
- Pode envolver diversas iterações
- Denição estável da arquitetura dos sub-sistemas
- Marco: Lifecycle Architecture Milestones (LCA)

Construção:

- Geração de código
- Descrever e desenvolver requisitos que ainda faltam
- Testes de sistema
- Marco: Initial Operational Capability (IOC)

Transição:

- Implementação do sistema no ambiente final
- Testes de aceitação e operação

- Migração de sistemas legados
- Treinamento
- Adaptações
- Marco: Product Release Milestone (PR)

Slide 4- Métodos Ágeis

- Menor ênfase em documentação
- Menor ênfase em big upfront design
- Envolvimento constante do cliente
- Novas práticas de programação
 - Testes, refactoring, integração contínua, etc

Extreme Programming (XP)

XP = VALORES + PRINCÍPIOS + PRÁTICAS

VALORES

- Comunicação
- Simplicidade
- Feedback
- Coragem
- Respeito
- Qualidade de vida

PRINCÍPIOS

- Economicidade
- Melhorias contínuas
- Falhas acontecem
- Baby Steps
- Responsabilidade pessoal

PRÁTICAS

Práticas sobre o Processo de Desenvolvimento	Práticas de Programação	Práticas de Gerenciamento de Projetos
Representante dos Clientes Histórias de Usuário Iterações Releases Planejamento de Releases Planejamento de Iterações Planning Poker Slack	Design Incremental Programação Pareada Testes Automatizados Desenvolvimento Dirigido por Testes (TDD) Build Automatizado Integração Contínua	Ambiente de Trabalho Contratos com Escopo Aberto Métricas

- Pair programming

CONTRATOS DE SOFTWARE

Escopo fechado:

- Cliente define requisitos
- Empresa desenvolvedora: preço + prazo

Escopo aberto:

- Escopo definido a cada iteração
- Pagamento por homem/ hora
- Contrato revogado a cada iteração

Scrum

- Não define práticas de programação pois não é apenas para projetos de software
- Processo mais rígido que o XP
- Reuniões diárias

SPRINTS

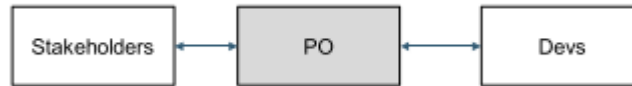
- Iterações
- Duração: de 15 dias a 1 mês

O que se faz em um sprint?

- Implementa-se algumas histórias dos usuários
- histórias = funcionalidades do sistema

Quem escreve as histórias?

- Product Owner (PO)
- Especialista no domínio do sistema



Backlog do produto

- Lista de histórias do usuários, que foram escritas pelo PO

Backlog do Sprint

- Lista de tarefas do sprint, com responsáveis e duração estimada
- Termina com dois eventos: review e retrospectiva

TIMES SCRUM

- 5 a 11 membros
 - 1 PO
 - 3 a 9 desenvolvedores
 - 1 scrum master

SCRUM MASTER

- Ajuda o time a seguir Scrum e seus eventos
- Remover de impedimentos não-técnicos
- Pode pertencer a mais de um time

REVISÃO DO SPRINT

- Time mostra o resultados e a implementação das histórias pode ser aprovada ou não
- Caso não seja aprovada a história volta para backlog do produto

RETROSPECTIVA

- Time se reúne para decidir o que melhorar, o que deu certo

time-box: As atividades têm uma duração bem definida

SCRUM BOARD

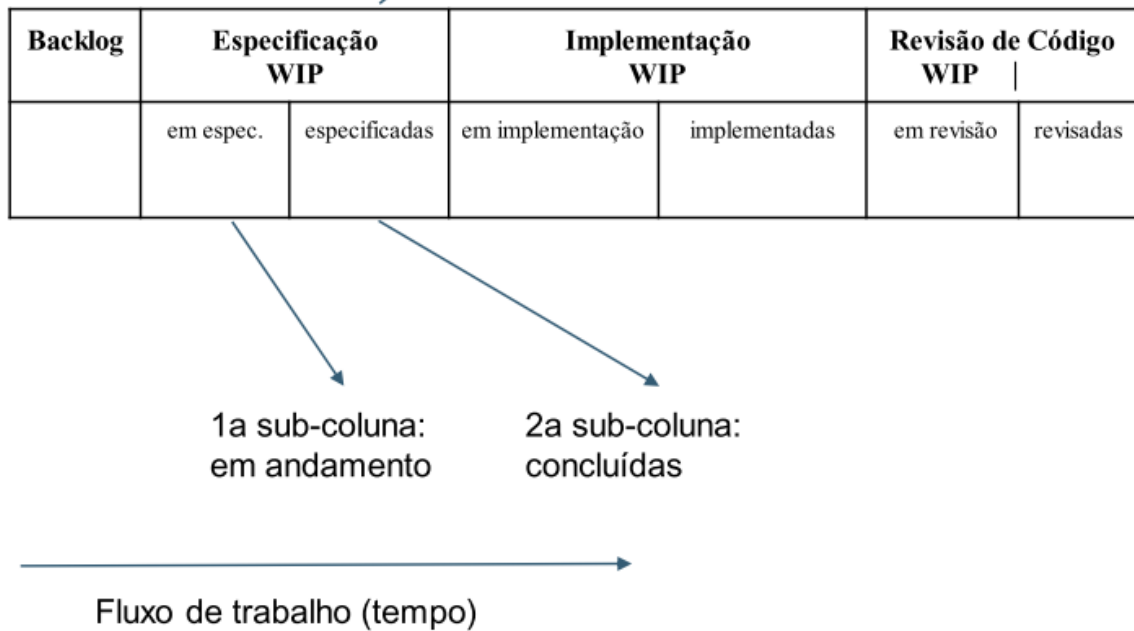
Backlog	To Do	Doing	Testing	Done
				

Burndown chart: Estimativa de tamanho de histórias de usuários

Story points: Unidade (inteiro) para comparação do tamanho de histórias.

Kanban

- Utiliza de um cartão visual
- Mais simples
- Não tem sprints
- Não é obrigatório usar papéis, eventos
- Time que define os papéis e eventos



Ideia central: sistema pull

- Membros “puxam” trabalho

Limetes Wip: Número máximo de tarefas em um passo

- Criar um fluxo de trabalho sustentável

Slide 5- Engenharia de requisitos (Processo unificado)

LEVANTAMENTO E ANÁLISE DE REQUISITOS

- O analista precisa utilizar todas as informações disponíveis e todas as fontes possíveis
- O levantamento de requisitos é o processo de descobrir quais são as funções que o sistema deve realizar e quais são as restrições que existem sobre essas funções
- Somente na fase de elaboração a análise dos requisitos será aprofundada

REQUISITOS FUNCIONAIS

- Descrição de uma função a ser executada pelo sistema
- Origem do requisito

- Quais as informações que são passadas do sistema para o usuário e vice-versa
- Restrições lógicas

REQUISITO NÃO-FUNCIONAIS

- Aparecem sempre ligados a requisitos funcionais e podem ser basicamente de dois tipos
 - Lógicos
 - Tecnológicos
- Classificados como de :
 - Usabilidade
 - Confiabilidade
 - Desempenho
 - Configurabilidade
 - Segurança
 - Implementação
 - Interface
 - Empacotamento
 - Legais

DOCUMENTO DE REQUISITOS

- Registra todos os tópicos relativos ao que o sistema deve fazer e sob que condições o sistema deve fazer as coisas
- Na fase de concepção, o documento não é totalmente estruturado e espera-se que seja completado em fases posteriores.

Slide 6- Engenharia de requisitos (Processo Ágeis)

Histórias de Usuários = 3C's

CARTÃO + CONVERSA + CONFIRMAÇÃO

Cartão: "lembrete" para conversar sobre o requisito durante o sprint

Confirmação: cenários que serão usados pelo PO para aceitar a implementação da história (escritos no verso do cartão)

EXEMPLO: LOJA VIRUTAL

- Cartão: "Fechar uma compra"
- **Conversas:** PO explica os meios de pagamento; as formas de entrega, formas de parcelamento, etc
- Confirmação:
 - Testar compra à vista e compra parcelado
 - Testar com cartões de crédito A, B e C
 - Testar com modos de entrega X e Y

HISTÓRIAS DE USUÁRIO

Exemplo: Sistema de controle de bibliotecas como usuário típico

- Como usuário típico, eu gostaria de realizar empréstimos de livros
- Como usuário típico, eu gostaria de devolver um livro que havia emprestado
- Como usuário típico, eu gostaria de renovar empréstimos de livros
- Como usuário típico, eu gostaria de pesquisar por livros

Características de boas histórias: Independentes, Abertas para Negociação, Agregar Valor, Estimáveis, Sucintas, Testáveis

REQUISITOS NÃO FUNCIONAIS

- Time deve definir RNF com o Product Owner
- Time deve incluir RNF nos critérios de conclusão de um sprint

Casos de uso

Transferir Valores entre Contas

Ator: Cliente do Banco

Fluxo normal:

- 1 - Autenticar Cliente
- 2 - Cliente informa agência e conta de destino da transferência
- 3 - Cliente informa valor que deseja transferir
- 4 - Cliente informa a data em que pretende realizar a operação
- 5 - Sistema efetua transferência
- 6 - Sistema pergunta se o cliente deseja realizar uma nova transferência

Extensões:

- 2a - Se conta e agência incorretas, solicitar nova conta e agência
- 3a - Se valor acima do saldo atual, solicitar novo valor
- 4a - Data informada deve ser a data atual ou no máximo um ano a frente
- 5a - Se data informada é a data atual, transferir imediatamente
- 5b - Se data informada é uma data futura, agendar transferência

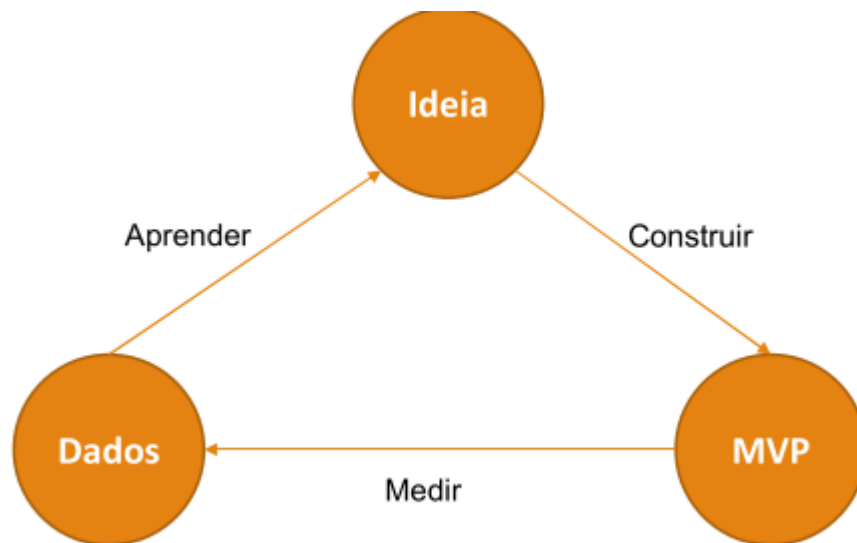
Slide 7- Engenharia de requisitos (validação)

Protótipos

- Geralmente utilizados para validação de telas
- Podem ser executáveis, apesar de ser incomum

Produto Mínimo viável (MVP)

- Sistema simples
- Conjunto de Requisitos Mínimos
- Suficiente para Testar a viabilidade de continuar
- **Objetivo:** Testar uma hipótese de negócio



- O aprendizado pode resultar em 3 decisões
 - Mais testes são necessário
 - Teste bem sucedido
 - Falhou após várias tentativas

Testes A/B

- Testes A/B são experimentos que comparam duas versões de algo, como um site ou aplicativo
- As versões são divididas em controle (original) e tratamento (nova proposta).
- Os usuários são aleatoriamente designados para uma das versões e suas interações são monitoradas.
- No final, os dados são analisados para determinar qual versão performa melhor em métricas específicas.
- Se a nova versão mostrar ganhos estatisticamente significativos, pode ser considerado migrar para ela.
- As empresas usam testes A/B para tomar decisões informadas sobre mudanças em seus produtos.
- Esses testes são amplamente utilizados por grandes empresas da Internet.
- Podem ser aplicados para testar diferentes MVPs, sistemas em si ou protótipos.

Slide 8- Gerenciamento de projetos

(Planejamento)

- O que é um projeto?
 - É um esforço temporário realizado para criar um produto ou serviço único

Etapas do gerenciamento de projetos

Iniciação → planejamento → Execução → Encerramento → ou → Monitoramento e controle

Planejamento e suas etapas

- Atividade fundamental para mitigar riscos

Especificar o escopo → Detalhar o escopo → Definir as atividades → Definir sequência das atividades → Estimar duração das atividades → Estimar custos das atividades → Definir cronograma → Definir o orçamento → Integrar planos

ESPECIFICAR O ESCOPO

- Escopo do Produto: característica e funcionalidades que o produto deve ter quando estiver pronto
- Opções para definição de escopo:
 - Documentos de visão (Processo Unificado)
 - Histórias de Usuário (Scrum)
 - Casos de uso
 - Cenários
 - Narrativa livre (Sumário Executivo)
 - Protótipos

DETALHAR O ESCOPO

- Quebrar o problema em problemas menores
- Estrutura analítica do projeto (EAP)
- Primeiro nível é o produto completo, já o segundo nível podem ser produtos parciais ou módulos do sistema e o demais níveis são decomposições dos segundo nível

DEFINIR ATIVIDADES

- Para cada pacote de trabalho de um EAP, definir:
 - Atividades necessárias para gerar o pacote
 - Recursos necessários para executar as atividades
- Exemplo: do pacote “Cadastro” de Usuários:
 1. Levantar requisitos
 2. Analisar requisitos
 3. Construir Protótipos
 4. Validar Protótipos
 5. Construir backend
 6. Efetuar testes Unitários
 7. Construir frontend
 8. Efetuar testes de Sistema

DEFINIR A SEQUÊNCIA DAS ATIVIDADES

- Estabelecer as dependências das atividades
 - Para executar a atividade M uma determinada atividade N precisar ser concluída

ESTIMAR DURAÇÃO DAS ATIVIDADES

- Cada atividade deve ter uma duração esperada

Estimativa via Planning Poker

- Toda a equipe participa no processo e é baseada em consenso
- Utilizado em métodos ágeis como o Scrum
- Cada membro da equipe recebe 13 cartas
 - Cada carta representa um quantitativo relativo a ser determinado pela equipe

- Processo: Os membros colocam suas estimativas de forma privada, revelam simultaneamente e calculam a média. Em seguida, discutem as diferenças entre as estimativas e refinam o consenso através de interações adicionais, se necessário

Estimativa Paramétrica

- Fórmulas obtidas por métodos de regressão a partir da execução de diversos projetos

COCOMO

- **Projetos simples:** fácil entendimento e equipe pequena
 - $\text{Esforço} = 2,4 \cdot \text{KLOC}^{1,05}$
 - $\text{Duração} = 2,5 \cdot \text{Esforço}^{0,38}$
- **Projetos de complexidade média:** experiência limitada da equipe
 - $\text{Esforço} = 3,0 \cdot \text{KLOC}^{1,12}$
 - $\text{Duração} = 2,5 \cdot \text{Esforço}^{0,35}$
- **Projetos complexos:** software crítico, interagindo com hardware
 - $\text{Esforço} = 3,6 \cdot \text{KLOC}^{1,20}$
 - $\text{Duração} = 2,5 \cdot \text{Esforço}^{0,32}$

APF

- Visa contar a quantidade de funcionalidades de um sistema

1. Contar elementos do software

- Número de Entradas Externas (EE):** conjunto de dados únicos que entram na fronteira do sistema – Ex.: tela de cadastro de produtos
- Número de Saídas Externas (SE):** conjunto de dados únicos que saem da fronteira do sistema – Ex.: relatório de vendas
- Número de Consultas Externas (CE):** combinação de entrada e saída onde a saída ocorre em função da entrada
- Número de Arquivos Lógicos Internos (ALI):** entidades únicas manipuladas pelo sistema – Ex.: entidade pedido
- Número de Arquivos de Interface Externos (AIE):** entidades compartilhadas por diferentes sistemas externos – Ex.: estoque sendo compartilhado pelos sistemas de

vendas e financeiro

1. Definir o nível de complexidade

- Definido em função da quantidade de campos e entidades envolvidas

3. Definir os pesos para cada elemento conforme complexidades

Elemento\Complexidade	Baixa	Média	Alta
Entradas Externas (EE)	3	4	6
Saídas Externas (SE)	4	5	7
Consultas Externas (CE)	3	4	6
Arquivos Lógicos Internos (ALI)	7	10	15
Arquivos de Interface Externos (AIE)	5	7	10

4. Obter Pontos de Função não Ajustados (PFNA)

- $PFNA = \sum \text{elemento} \times \text{Peso}$

5. Ajustar os pontos de função (PF)

6. Obter Pontos de Função Ajustados (PF)

- $PF = PFNA \times (0,65 + 0,01 \times \sum \text{Respostas})$

7. Converter PFNA ou PF em LOC

Linguagem	LOC	Linguagem	LOC	Linguagem	LOC
Assembly	320	FORTRAN 77	107	Prolog	64
C	128	Java	53	Shell Script	107
C++	55	PASCAL	91	Visual Basic 5	29
COBOL	91	PERL	27	Visual C++	34

Fórmula de KLOC = (total de PFNA * LOC por PFNA) / 1000

Estimativa por combinação

- Coleta de estimativas de tempo usando diferentes técnicas
- Cálculo de tempo esperado (TE)

DEFINIR CRONOGRAMA

- O que deve ser feito, em que ordem deve ser feito e quanto tempo leva para fazer

DEFINIR ORÇAMENTO

- Deve considerar a margem de lucro ou aumentar a probabilidade de ganhar o projeto

INTEGRAR PLANOS

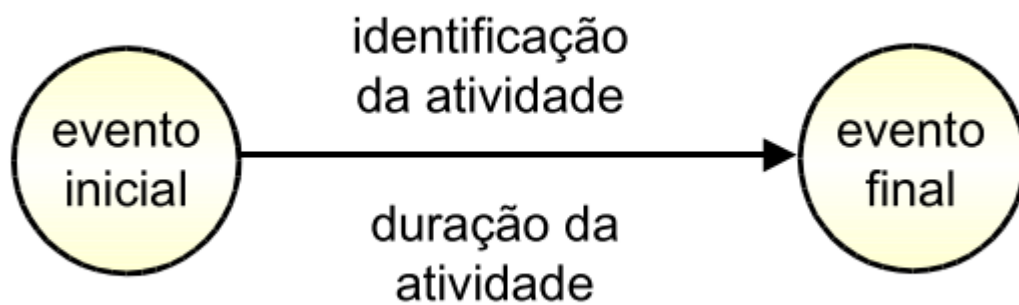
- O plano do projeto precisa ser integrado com demais documentos conforme o modelo de processo de software adotado

side 9- Gerenciamento de Projetos (Controle de Tempo)

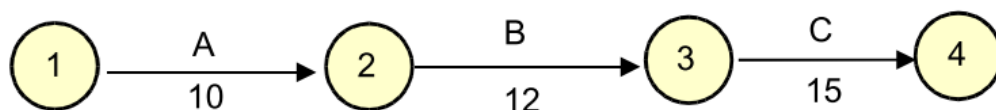
Redes Pert-CPM ou Redes de Planejamento

- É a representação gráfica de um programa, na qual se apresenta a sequência lógica do planejamento com as interdependências das tarefas, tendo por finalidade atingir um objetivo

Representação da rede



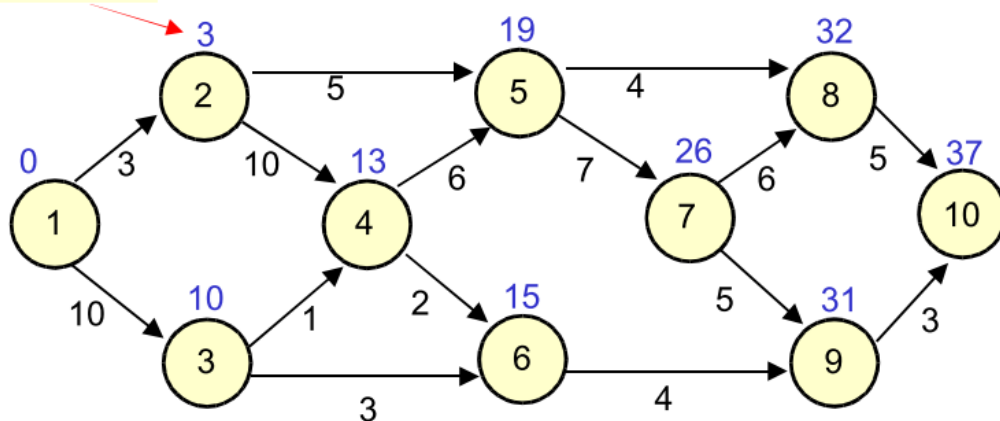
TEMPO DE EXECUÇÃO DA REDE



$$\text{Tempo de Execução} = 10 + 12 + 15 = 37$$

CEDO DO EVENTO

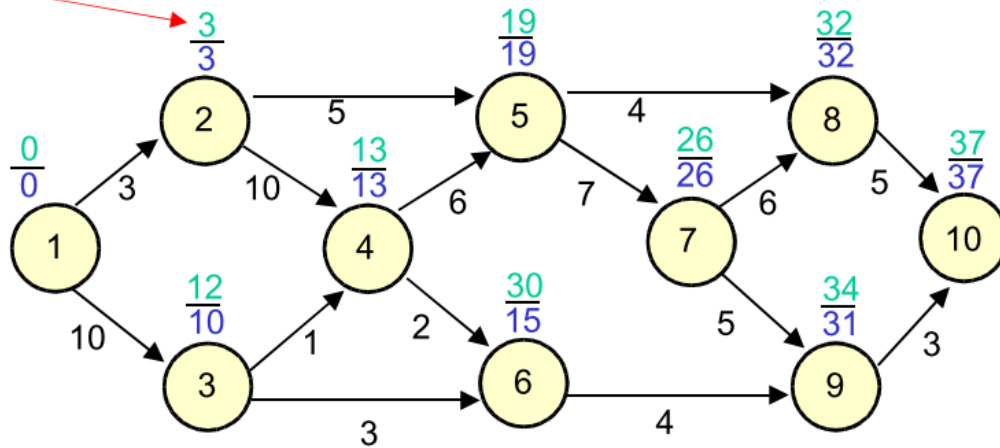
cedo do evento : tempo necessário para que o evento seja atingido, sem atrasos



$$\text{cedo} = \max (\text{cedo anterior} + \text{duração})$$

TARDE DO EVENTO

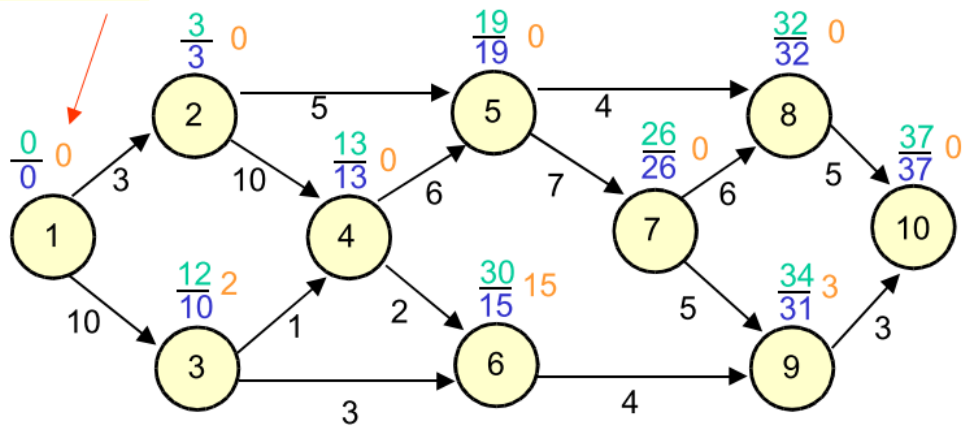
tarde do evento : tempo limite para a realização do evento para não atrasar o projeto



$$\text{tarde} = \min (\text{tarde posterior} - \text{duração})$$

FOLGA DO EVENTO

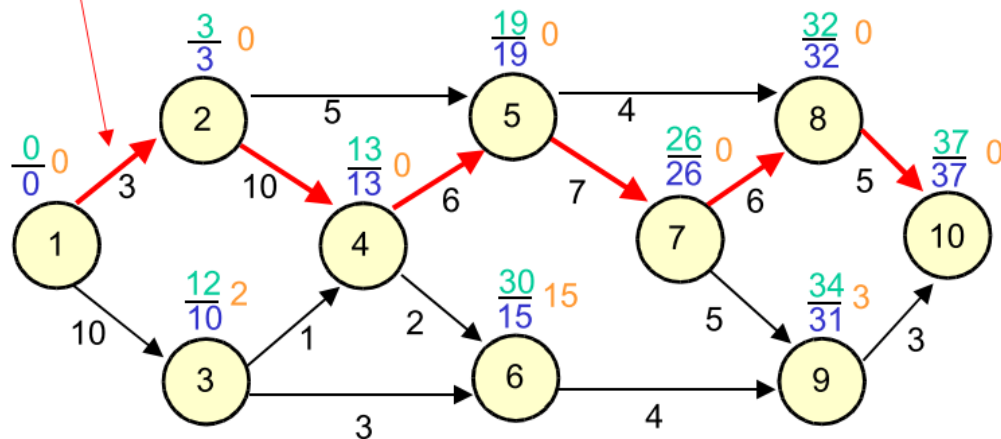
folga do evento : diferença entre o tarde e o cedo do evento



$$\text{folga} = \text{tarde} - \text{cedo}$$

CAMINHO CRÍTICO

caminho crítico : vai do evento inicial ao evento final, passa pelas atividades de maior duração e a soma das durações corresponde ao cedo do evento final (os eventos por onde ele passa têm a menor folga).



Tarde = Cedo não existe um prazo para elaboração do projeto

Tarde < Cedo existe um prazo que é menor do que o prazo normal para elaboração do projeto

Tarde > Cedo existe um prazo que é maior do que o prazo normal para elaboração do projeto

side 10- Gerenciamento de Projetos (Monitoramento e Controle)

- **Monitorar** as atividades previstas no plano do projeto
- Caso o andamento da execução do projeto desvie do estimado, é preciso **controlar** esse desvio para atender os objetivos do projeto

Técnicas:

- Quadro Kanban (Métodos Ágeis)
- Gráficos de burndown e burnup (Métodos Ágeis)
- Análise de valor agregado (Métodos Clássicos)

Gráfico burndown

- Tem como foco a Sprint corrente
- Foco em mostrar atividades pendentes

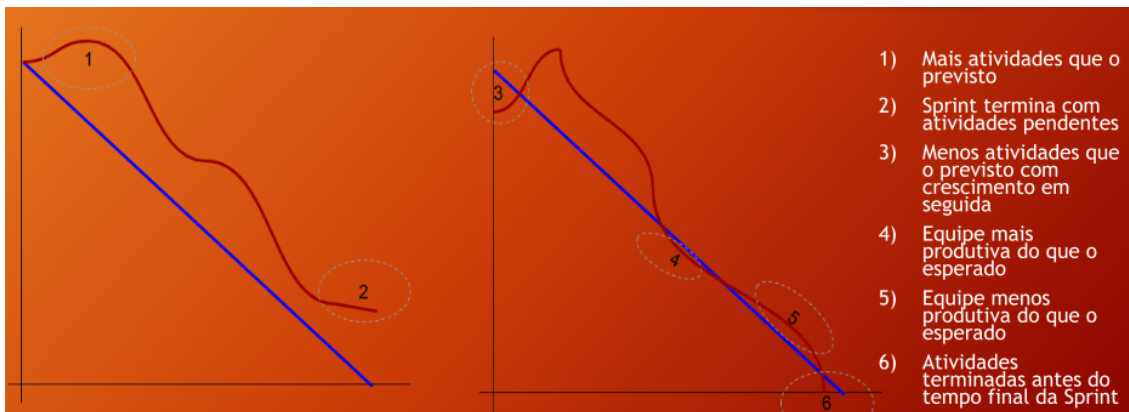
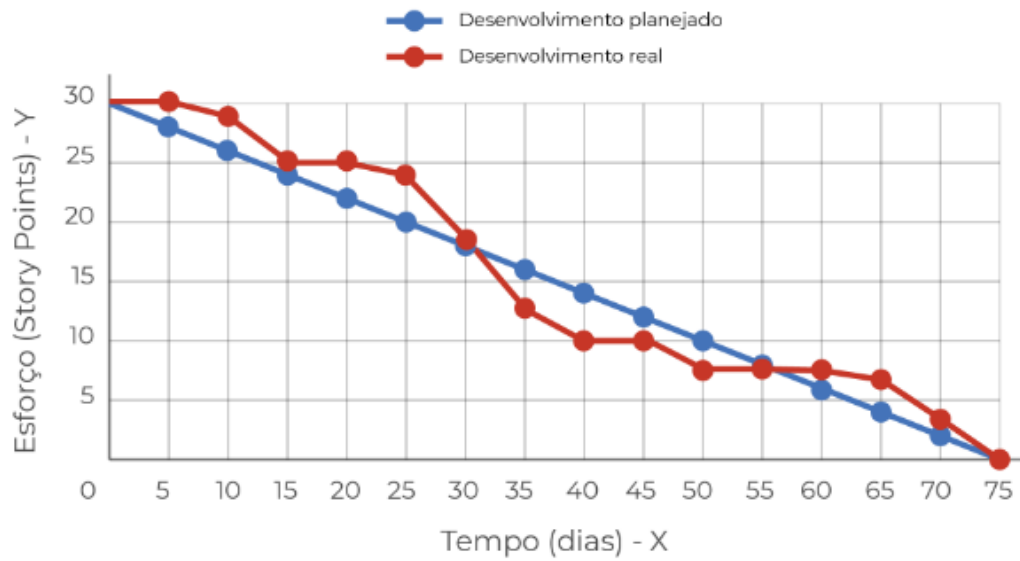
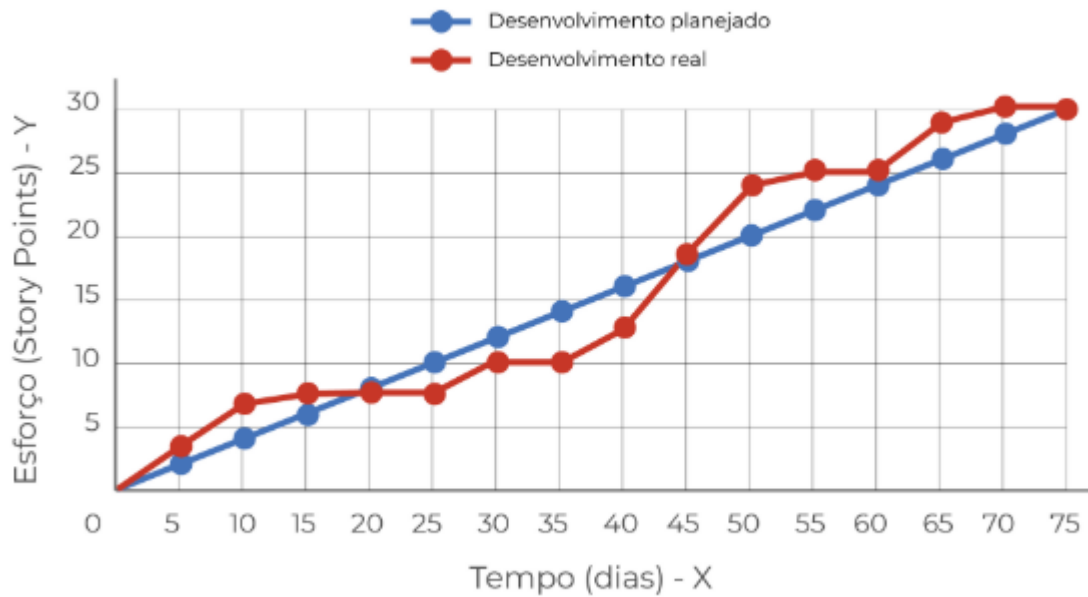


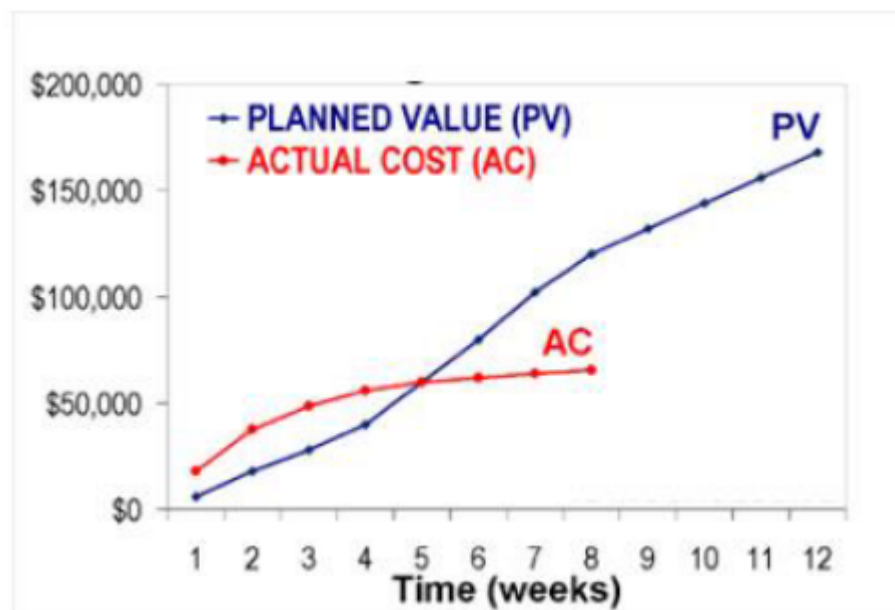
Gráfico Burnup

- Tem como foco a Sprint corrente
- Foco em mostrar atividades realizadas



Análise do valor agregado

- Considera a dimensão de custo



Cálculo do valor planejado

PV (Valor planejado)

P%C (Porcentual planejado de conclusão)

BAC (Custo total)

$$PV = BAC \cdot P\%C$$

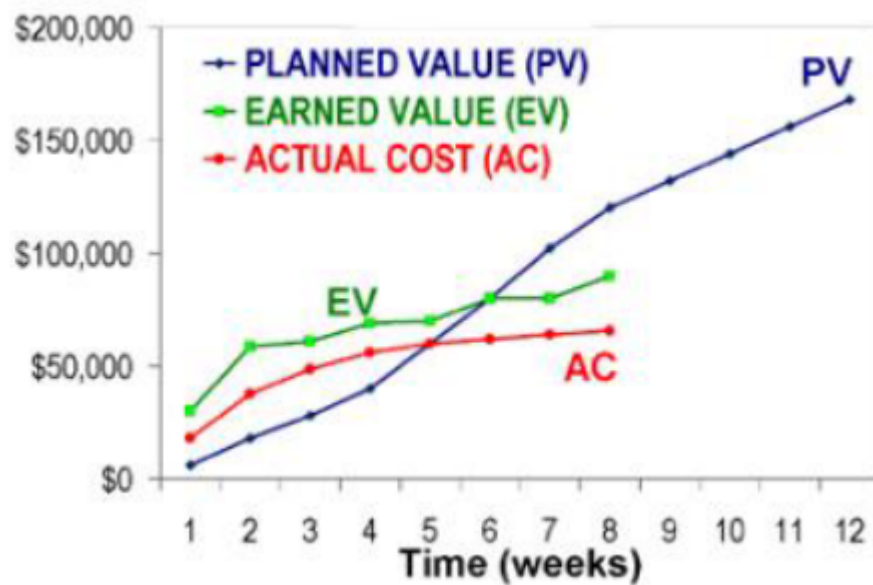
Cálculo do valor agregado

EV (Valor agregado)

A%C (Porcentual real de conclusão)

BAC (Custo total)

$$EV = BAC \cdot A\%C$$



- $EV < PV$ → Projeto está atrasado
- $EV > AC$ → Projeto está abaixo do custo

Análise do valor agregado

DESEMPENHO DO CRONOGRAMA

SPI (Schedule Performance Index) → $SPI = EV / PV$

- $SPI = 1$ → no cronograma
- $SPI > 1$ → adiantado
- $SPI < 1$ → atrasado

SV Schedule Variance (variação de cronograma)

- $SV = EV - PV$

DESEMPENHO DE CUSTO

CPI : Cost Performance Index → $CPI = EV / AC$

- $CPI = 1$ → no custo
- $CPI > 1$ → abaixo do custo
- $CPI < 1$ → acima do custo

CV: Cost Variance (variação de custo)

- $CV = EV - AC$

Slide 11- Princípios de projeto

- O problema mais fundamental em ciência da computação é a tarefa de decomposição de problemas: como dividir um problema complexo em partes que possam ser resolvidas de forma independente.
- Definição de projeto: Quebrar um “problema grande” em partes menores.

Propriedades de Projeto

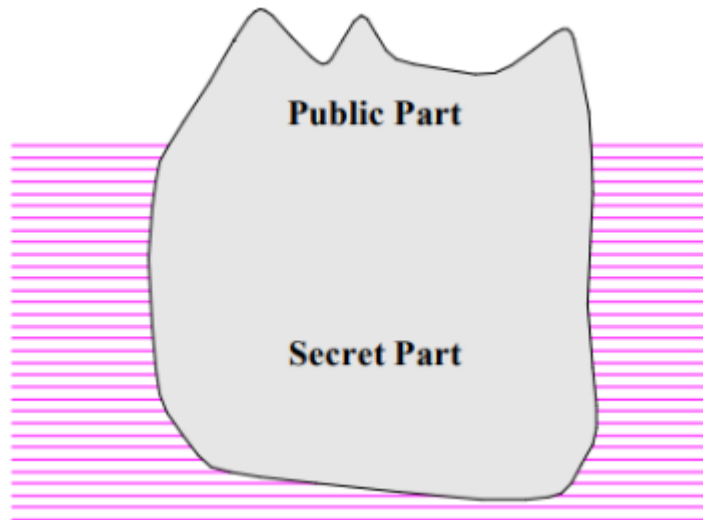
INTEGRIDADE CONCEITUAL

- Coerência e padronização de funcionalidades, projeto e implementação.
- Decisões de um sistema devem ser coerentes (seguir padrões)
- Integridade conceitual vale também para o projeto e código de um sistema, exemplos:
 - Todas as variáveis seguem o mesmo padrão de nomes
 - Todas as páginas usam o mesmo framework
- A integridade conceitual é a consideração mais importante no projeto de sistemas, pois facilita uso e entendimento de um sistema

OCULTAMENTO DE INFORMAÇÕES

- Classes devem ocultar detalhes internos de sua implementação (usando modificador `private`)
 - Principalmente aqueles sujeitos a mudanças

- Prática de esconder detalhes internos de implementação, promovendo a encapsulação e reduzindo dependências diretas.



COESÃO

- Cada módulo ou classe deve ter uma única responsabilidade ou função.

ACOPLAMENTO

- Refere-se à interdependência entre módulos; deve ser minimizado e controlado para evitar dependências frágeis.
- Nenhuma classe é uma ilha ...
- Classes dependem umas das outras (chamam métodos de outras classes, estendem outras classes, etc)
- A questão principal é a **qualidade** desse acoplamento
- **Acoplamento estático (ou estrutural):** ◻
 - Classe A depende de uma classe B ◻
 - No código de A, existe uma referência explícita para B ◻
 - Pode ser um acoplamento bom ou ruim
- **Acoplamento evolutivo (ou lógico):** ◻
 - No código de A, não existe uma referência para B ◻

- Porém, mudanças em B podem impactar a classe A ▮
- Tipo de acoplamento sempre ruim

▮ Maximize a coesão, minimize o acoplamento.

Slide 12- Princípios de projeto

(01)Princípio da Responsabilidade Única

- Cada classe deve ter apenas uma única responsabilidade

Vantagens: Facilita a manutenção e reduz a complexiidade do código

(02)Princípio da Segregação de Interfaces

- Interfaces devem ser pequenas, coesas e específicas para cada tipo de cliente

Exemplo Prático: Interfaces separadas para funcionários CLT e públicos ao invés de uma interface genérica.

(03)Princípio da Inversão de Dependências

- Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações (interfaces)

▮ Prefira interfaces a Classes

(04)Princípio Composição a Herança

- Utilize composição em vez de herança para reutilizar código

▮ Não force o uso de herança

(05)Princípio de Demeter

- Evite longas “cadeias” de chamadas de métodos
- Exemplo: `obj.getA().getB().getC().getD().getOqueEuPreciso()`

(06)Princípio aberto/fechado

- Uma classe deve estar fechada para modificações, mas aberta para extensões

(07)Princípio de Substituição de Liskov

- Objetos de uma classe base devem ser substituíveis por objetos de subclasses sem afetar a integridade da aplicação
-

side 13- Princípios de projeto

Métricas de Código-Fonte

TAMANHO

- Medido em linhas de código (LOC)
- Importante especificar se inclui linhas em branco e comentários
- Não deve ser usada para medir a produtividade dos programadores

COESÃO

- Medida pela métrica LCOM (Lack of Cohesion Between Methods)
- Quanto mais alta, pior é a classe
- Calculada comparando conjuntos de atributos acessados por pares de métodos
- Para uma classe ser coesa, os métodos compartilham pelo menos um atributo

ACOPLAMENTO

- Medida pela métrica CBO (Coupling Between Objects).
- Conta o número de classes das quais uma classe depende sintaticamente ou estruturalmente
- Exemplos: se a classe chama métodos de outra classe, acessa seus atributos públicos ou herda dela.

COMPLEXIDADE

- Medida pela complexidade ciclomática
- É o número de caminhos independentes através do código

side 14- Padrões de projeto

- Os padrões de projeto são técnicas para resolver problemas comuns em design de software
- Eles fornecem uma solução esquematizada que ajuda a resolver problemas de design de software de maneira eficiente e eficaz.

Três categorias principais: Criacionais, Estruturais e Comportamentais.

Criacionais	Estruturais	Comportamentais
Abstract Factory Factory Method Singleton Builder Prototype	Proxy Adapter Facade Decorator Bridge Composite Flyweight	Strategy Observer Template Method Visitor Chain of Responsibility Command Interpreter Iterator Mediator Memento State

(01) Fábrica (Criacional)

- É usado para criar uma instância de várias classes derivadas
- Por exemplo, uma fábrica de conexões de rede poderia retornar uma conexão TCP ou UDP dependendo de parâmetros de entrada. Centraliza a criação de objetos, facilitando mudanças no processo de instânciação.

```
class ChannelFactory {
    public static Channel create() { // método fábrica estático
        return new TCPChannel();
    }
}
```

Único ponto de mudança se quisermos usar UDP

Sem Fábrica

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

⇒

Com Método Fábrica Estático

```
void f() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void g() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void h() {  
    Channel c = ChannelFactory.create();  
    ...  
}
```

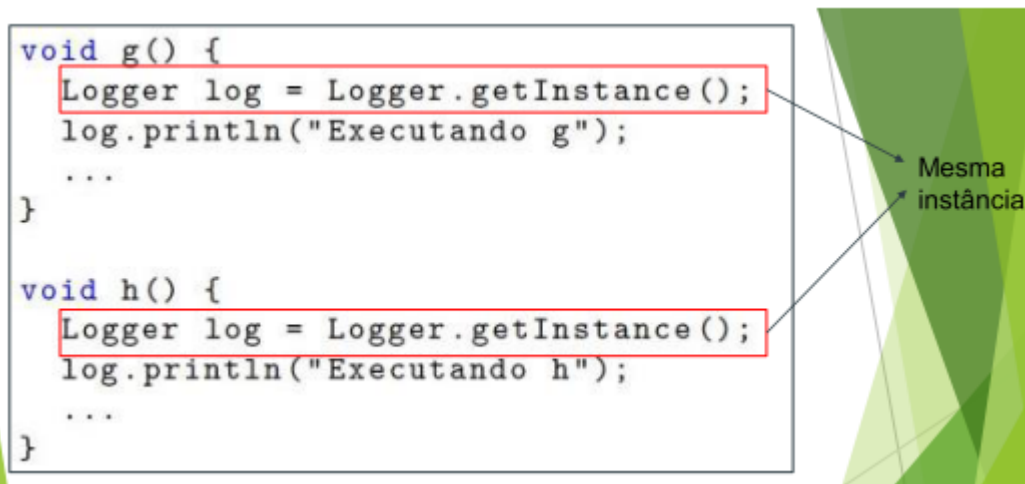
(02) Singleton (Criacional)

- Garante que uma classe tenha apenas uma instância e fornece um ponto de acesso global a essa instância

```
void f() {  
    Logger log = new Logger();  
    log.println("Executando f");  
    ...  
}  
  
void g() {  
    Logger log = new Logger();  
    log.println("Executando g");  
    ...  
}  
  
void h() {  
    Logger log = new Logger();  
    log.println("Executando h");  
    ...  
}
```

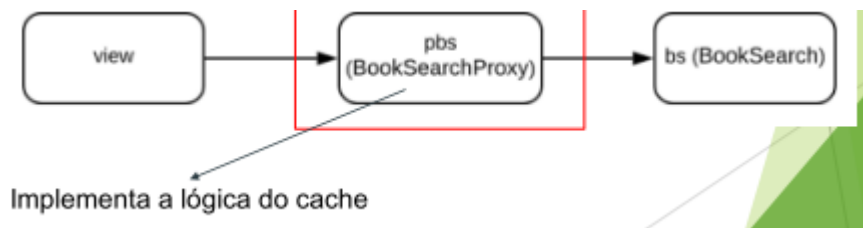
Problema:

Cada método cliente usa sua própria instância de Logger



(03) Proxy (Estrutural)

- Fornece um substituto ou representante de outro objeto para controlar o acesso a ele
- Usado frequentemente em situações onde objetos precisam de um intermediário ou quando funcionalidades adicionais são necessárias quando acessando um objeto particular



(04) Adaptador (Estrutural)

- Converter a interface de uma classe em outra interface que os clientes esperam
- Permitem que classes trabalhem juntas, apesar de interfaces incompatíveis

(05) Fachada (Estrutural)

- Fornece uma interface unificada para um conjunto de interfaces em um subsistema
- Define uma interface de nível mais alto que torna o subsistema mais fácil de ser usado

(06) Decorador (Estrutural)

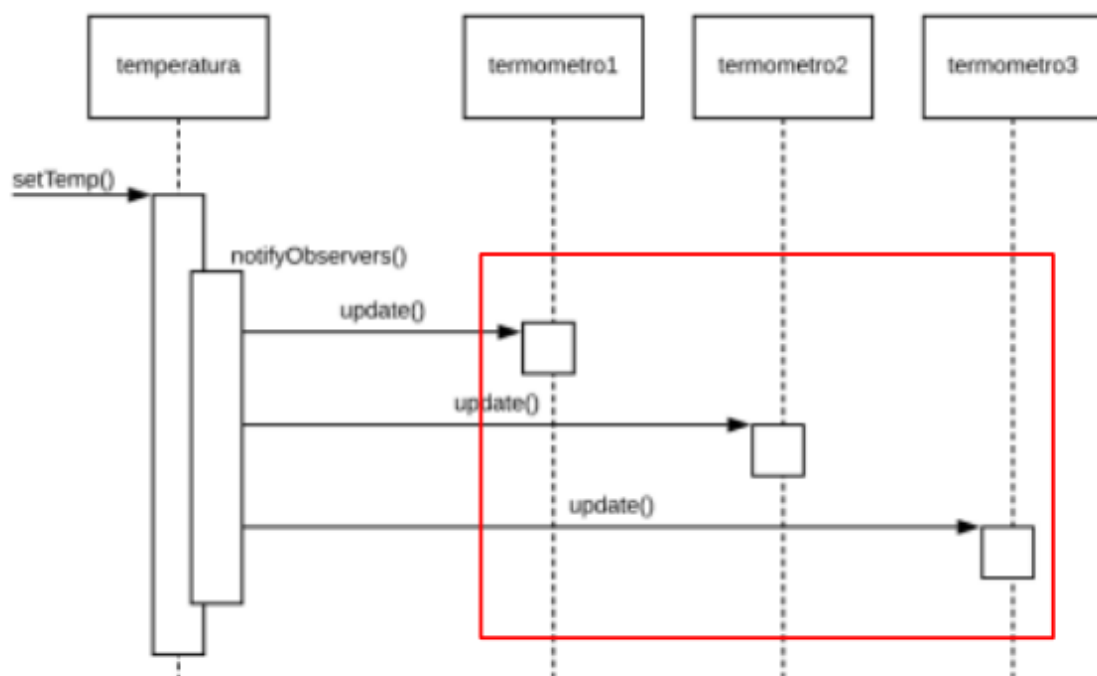
- Adiciona responsabilidades adicionais a um objeto dinamicamente
- Fornecem uma alternativa flexível ao uso de subclasses para extensão de funcionalidades
- Por meio da composição e não mais por meio de herança

(07) Strategy(Comportamentais)

- Parametrizar os algoritmos usado por uma classe
- Tornar uma classe “aberta” a novos algoritmos

(08)Obeservador(Comportamentais)

- Um modo de notificação de mudança a vários objetos para garantir a consistência entre classes estreitamente cooperativas
- É frequentemente usado em interfaces gráficas para informar e atualizar automaticamente outras partes de um aplicativo quando um objeto muda



(09)Template Method(Comportamentais)

- Define o esqueleto de um algoritmo em uma operação, postergando alguns passos para subclasses
- Template Method permite que subclasses redefinam certos passos de um algoritmo sem mudar a estrutura do algoritmo

Quando não usar padrões de projeto?

- Se a chance de mudanças é zero, não precisamos de padrões de projeto